

138. Un record qui survit à l'extinction

La cartouche a une pile, et de la mémoire qui reste.

1. Ce qu'il faut comprendre

`sauver(case, valeur)` et `sauvegarde(case)` écrivent et relisent la **mémoire sauvegardée de la cartouche** — celle qui survit à l'extinction.

Sur une vraie console, c'est une pile qui la tient. Dans la page, c'est le navigateur ; le programme, lui, ne voit aucune différence.

On ne sauve **que lorsque le record change**. Écrire à chaque image userait la mémoire d'une vraie cartouche, et n'apporterait rien.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sauvegarde>` → [119.2. La fonction sauvegarde\(\) — relire un nombre gardé](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sauver>` → [119.1. La fonction sauver\(\) — garder un nombre, même éteint](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **691 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         159 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles   58 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
LireManette    37 o ← bouton() — lit les huit boutons
Tuiles         256 o ← CopierTuiles — 16 lettres sur 43
"RECORD"       6 o ← texte() — un texte gravé
"SCORE"        5 o ← texte() — un texte gravé
```

Lettres gravées : CDEORS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre()      x 3    30 o sur place, dans main() + EcrireNombre (33 o)
sauver()       x 1    24 o sur place, dans main()
texte()        x 2    22 o sur place, dans main() + EcrireTexte (10 o)
```

```
sauvegarde() × 1    20 o sur place, dans main()
bouton()      × 1    12 o sur place, dans main() + LireManette (37 o)
image()       × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : chaque appui sur A donne un point. Le meilleur score (le RECORD)
// est gardé dans la cartouche : il est encore là après avoir éteint la console.
//
//  sauvegarde(numéro)      : relit une case de la mémoire de la cartouche.
//  sauver(numéro, valeur)  : écrit dans cette case.

#include <sauvegarde>    // relit un nombre gardé dans la cartouche
#include <texte>         // écrit un texte à l'écran
#include <nombre>        // écrit un nombre en chiffres
#include <bouton>        // lit un bouton de la manette
#include <sauver>        // garde un nombre dans la cartouche, même éteinte
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    uint8_t record = sauvegarde(0);          // On relit le record gardé dans la case 0.
    uint8_t score = 0;
    uint8_t avant = 0;                      // A au tour d'avant.

    texte(1, 2, "RECORD");
    nombre(10, 2, record);

    while (true) {                          // Répète pour toujours :
        uint8_t appui = bouton(A);
        if (appui && !avant) score++;       // Début d'un appui : un point.
        avant = appui;

        texte(1, 6, "SCORE");
        nombre(10, 6, score);

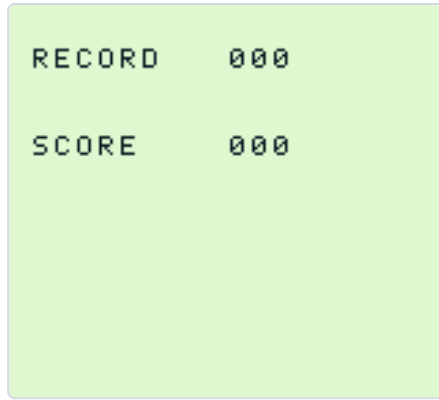
        if (score > record) {               // Record battu ?
            record = score;
            sauver(0, record);              // on l'écrit dans la cartouche,
            nombre(10, 2, record);          // et on l'affiche.
        }
        image();                            // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



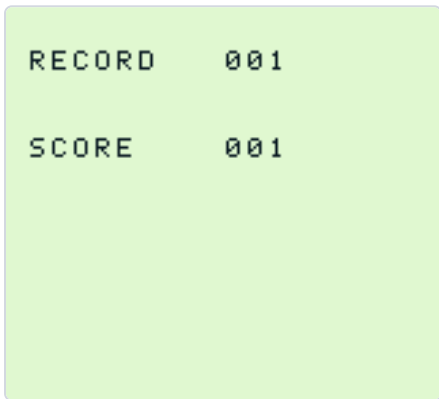
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Le record suit le score dès qu'il le dépasse.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les étiquettes sont là
- ✓ trois appuis font trois points (3)
- ✓ le record a suivi (3)
- ✓ et il est affiché (au départ 000)

Ensuite : **139. Compter ce que coûte une boucle** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++